# Artificial Intelligence
# Introduction and examples

Stefaan Haspeslagh
Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

Artificial Intelligence: introduction and examples

# BLIND SEARCH 2

## Blind search

- We have:
  - 2 algorithms to ‘search the tree’
    - Breath-first
    - Depth-first
  - None of both are ‘the best’
- Can we do better?
  - Non-deterministic search
  - Iterative deepening

In het vorige topic hebben we al aangehaald dat we beter kunnen doen dan met de algoritmen diepte- en breedte-eerst.
In dit (korte) topic stellen we twee algoritmen voor:
- non-deterministic search
- iterative deepening

## Non-deterministic search

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths in random places in QUEUE

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

Bij diepte- en breedte eerst voegen we nieuwe paden respectievelijk aan het begin of aan het einde van de QUEUE in.

Bij non-deterministic search gaan we die nieuwe paden gewoon willekeurig in de QUEUE invoeren.

## Iterative deepening search

- Restrict a depth-first search to a fixed depth.
- If no path was found, increase the depth and restart the search.

Non deterministic search gaat kriskras doorheen de zoekboom zoeken. Met iterative deepening proberen we een iets ‘stabieler’ algoritme voorop schuiven.
Het probleem in het vorig topic was dat we moeilijk konden kiezen tussen depth-first and breadth-first search, want beiden hadden voor- en nadelen
Bij iterative deepening proberen we de voordelen van beide te combineren. We gaan diepte-eerst toepassen, maar enkel tot op een bepaalde diepte.
Indien we op een bepaalde diepte geen oplossing gevonden hebben, proberen we 1 niveau verder te zoeken…
We starten dus bij niveau 1, passen diepte-eerst toe. Indien we geen oplossing gevonden hebben, starten we opnieuw diepte-eerst, maar nu tot op niveau 2. We gaan zo verder tot we een oplossing gevonden hebben, of tot de volledige boom doorzocht werd (en er dus geen oplossing is).

In de volgende slides schrijven we deze werkwijze neer in een algoritme.

## Iterative deepening search: depth-limited search

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - IF path has length smaller than DEPTH:
     - Create new paths (to all children);
     - Reject the new paths with loops;
     - Add the new paths to the **front** of QUEUE

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

## Iterative deepening search

1. DEPTH = 1

2. WHILE goal is not reached
   - perform Depth-limited search
   - DEPTH = DEPTH + 1

## Iterative deepening search: the BEST blind search

- Complete:
  - YES!
  - Even finds the shortest path (similar to breadth-first)
- Memory:
  - O(b*m)
- Speed:
  - If the path is found for Depth = m, then how much time was waisted constructing the smaller trees? $1 + \ldots + b^{m-2} + b^{m-1} = (b^m-1) / (b-1) = O(b^{m-1})$
  - While the work spent at DEPTH = m itself is $O(b^m)$

  → VERY GOOD trade-off!

**Completeness**

Net zoals breedte-eerst is het algoritme compleet. Door telkens slechts tot op een bepaalde diepte te zoeken, kunnen we niet in een oneidige tak terecht komen. Het algoritme vindt zelfs het kortste pad, net zoals breedte-eerst.

**Memory**

Iterative deepening is depth-first tot op diepte m. Dus kunnen we hier de redenering toepassen die we bij DFS hebben toegepast, alleen gaan we nu tot de minst diepe oplossing m ipv tot de volledige diepte d van de zoeekboom. Zo krijgen we complexiteit O(b*m).

**Speed**

Het algoritme bestaat dus uit het meerdere keren uitvoeren van diepte eerst tot op een bepaalde hoogte. We gaan telkens ervan uit dat op elk niveau alle knopen moeten bekeken worden, tot op niveau m waar de minst diepe oplossing (doelnode) zich bevindt.
Centrale vraag is hoeveel werk we teveel gedaan hebben door eerste kleinere bomen te doorzoeken i.p.v. onmiddellijk te zoeken tot op niveau m. (Let wel: onmiddellijk zoeken tot op niveau m is niet mogelijk, want het getal m is niet gekend bij een

willekeurige zoekboom. Pas achteraf, als de goal gevonden is weet men wat m is.) Het bewijs staat in het boek, maar moet je niet kennen. Aangezien tot op het niveau m-1 er $O(b^{m-1})$ werkt verricht is, en aangezien $O(b^{m-1})$ significant, om niet te zeggen veel kleiner is dan $O(b^m)$, valt de schade nogal mee. Iterative deepening vergt meer werkt dan breedte-eerst op zich, maar aangezien het geheugengebruik beter is, is dit een zeer goede trade-off.

Iterative deepening is eigenlijk het enige praktisch bruikbare blind search algoritme dat we gezien hebben.

## Finally: what blind search algorithm to choose?

| Criterium | DFS | BFS | IDS |
|---|---|---|---|
| **Complete?** | Yes, if<br>(1) no infinite branches<br>(2) no cycles | **Yes, always** | **Yes, always** |
| **Shortest path?** | No, maybe by accident | **Yes, always** | **Yes, always** |
| **Speed?** | $O(b^d)$ | $\mathbf{O(b^m)}$ | $\mathbf{O(b^m)}$ |
| **Memory?** | $O(bd)$ | $O(b^m)$ | $\mathbf{O(bm)}$ |

Iterative Deepening Search!

In de tabel staat een samenvatting van de analyse van de zoekalgoritmen waarvan we de complexiteit hebben onderzocht.

IDS scoort overal het best. We weten dat BFS het op vlak van rekentijd nog net iets beter zal doen, maar het geheugengebruik bij IDS is veel beter, want O(b*m) << $O(b^m)$. Dus die extra berekeningen door het itereren over verschillende dieptes zijn over het algemeen de moeite waard!

# Questions

?

Coordinates:

Dr. Stefaan Haspeslagh

A.I. Research & Education

Vives University of applied sciences – KU Leuven Association

E-mail: stefaan.haspeslagh@vives.be